# ICD-10 Category Codification from Clinical Literals using Classical NLP Baselines and Biomedical RoBERTa

Fundamentals of Natural Language / NLP-I – UAB-ASHO AI Codification Project

**Phoebe Iglesias** (1713459) **David Redrejo** (1790336) **Pau Rossell** (1750424)

Supervisors: Ernest Valveny and Lei Kang Academic year 2025–2026

BSc Artificial Intelligence, Universitat Autònoma de Barcelona

Repository: https://github.com/david06redrejo-bot/clinical-literal-icd-coding

Competition: https://www.kaggle.com/competitions/uab-asho-ai-codification

**Abstract**

This report describes our solution for the Kaggle competition *UAB-ASHO AI Codification*. The task is to predict exactly one ICD-10 category prefix, `y_category`, from a short clinical literal. We built the project as a reproducible sequence of decisions rather than a single model jump: first data and annotation analysis, then conservative preprocessing, classical vector-space baselines, similarity and catalogue analysis, biomedical Spanish RoBERTa fine-tuning, imbalance-aware variants, ablation studies, error analysis, and finally a diverse ensemble. Our best verified public Kaggle score in the repository is 0.587, obtained with `v10_vote_diverse_no_retrieval`. The private/final competition ranking is not claimed because no verified evidence is stored in the repository yet. Beyond the score, the project helped us connect course concepts—corpora, regular expressions, tokenization, TF–IDF, Transformers, pretrained language models, evaluation, error analysis, and ensembles—to a realistic healthcare language problem where each modelling choice has practical consequences.

## 1. Introduction: Why ICD Coding Matters

ICD coding is one of those hospital processes that is easy to underestimate until we see what depends on it. Codes support clinical documentation, statistics, diagnosis-related groups (DRGs), reimbursement, hospital management, audits, and longitudinal analysis of care. Manual coding requires expertise and attention: clinical language is abbreviated, incomplete, and often written under pressure. Errors can affect not only data quality, but also administrative decisions and financial flows.

Automated ICD coding is therefore attractive, but risky. A model can help by ranking or suggesting likely categories, yet it should not replace professional coders. Our project focuses on a narrower competition task: given a clinical literal, predict the first character of the ICD code. This is much simpler than full ICD coding from complete electronic medical records. After this first observation, however, the task did not become trivial: short literals often lack context and may contain abbreviations, punctuation, accents, digits, and domain-specific shorthand.

## 2. Project Context and Competition

The project was developed for Fundamentals of Natural Language / NLP-I at Universitat Autònoma de Barcelona during academic year 2025–2026. The team is Team 10: Phoebe Iglesias, David Redrejo, and Pau Rossell. The supervisors/professors are Ernest Valveny and Lei Kang.

The competition was *UAB-ASHO AI Codification* on Kaggle [1]. The expected output is a single label `y_category` per leaderboard row. Kaggle public leaderboard feedback was useful for checking submissions, but we kept internal validation results separate from public scores to avoid telling a misleading story. After an initial baseline and short presentation, we had a follow-up with Lei Kang, who told us the direction was good. At that stage, after baseline tests, the team was around second in the competition. We include this as project context; a formal final ranking claim should only be made after adding verified evidence such as a screenshot or official table.

## 3. Data and Annotation Analysis

Our first technical phase was deliberately *Analyzing the data and the annotations*. This follows the Data Engineering mindset: before modeling, inspect the data-generating process, schemas, missingness, duplicates, distributions, and possible leakage. Table 1 summarizes the actual files found.

**Table 1:** Dataset files found in `data/raw`.

| File | Rows | Cols | Role |
|---|---|---|---|
| `codification_data.csv` | 13,700 | 2 | Training literals with `Code`. |
| `leaderboard_data.csv` | 6,667 | 2 | Kaggle leaderboard literals. |
| `icd_d_p_pairs.csv` | 179,742 | 3 | ICD catalogue pairs/descriptions. |

The annotation contract is:

$$y_i = \text{firstchar}(\text{str}(\text{Code}_i)). \tag{1}$$

In code, this is:

```
df["y_category"] = df["Code"].astype(str).str[0]
```

The training set contains all 36 expected categories. The leaderboard file does not contain `y_category`, which is normal for an unlabeled Kaggle test file even though the initial statement mentioned it.

The full EDA found 4,059 unique ICD codes in training and 179,742 ICD codes in the catalogue. It also found 1,668 duplicate literal groups with multiple full ICD codes, and 1,486 duplicate literal groups with multiple `y_category` labels. Examples such as *obesidad parto*, *Fumadora part*, and *Hipotiroidismo parto* appeared with several different codes and categories. This changed how we approached modeling: identical or nearly identical literals could not be treated as automatically solved string matches.

Figure 1 is the first reason why the project cannot be evaluated with accuracy alone. Categories such as `Z`, `O` and `0` dominate the training set, while several labels have far fewer examples. This means that a model can look

acceptable by learning the frequent labels and still fail on clinically meaningful rare categories. Figure 2 complements this view at the full-code level: the number of examples per ICD code decreases sharply, which is typical of clinical coding datasets and explains why exact code prediction would be much harder than prefix prediction.

Table 2 records the main EDA numbers used later in the modeling decisions. The table is not only descriptive; it becomes the reason for later choices. The duplicate and ambiguity counts justify careful validation, the long-tail distribution motivates macro-F1, and the train–leaderboard overlap explains why lexical baselines were worth testing before moving to Transformers.

**Table 2:** Key EDA values used in the report.

| Observation | Value |
|---|---|
| Training rows | 13,700 |
| Leaderboard rows | 6,667 |
| Unique training ICD codes | 4,059 |
| Unique `y_category` labels | 36 |
| Duplicate literal groups | 1,668 |
| Same literal, multiple categories | 1,486 |
| Leaderboard normalized literals seen in train | 51.8% |
| Mean train literal length | 16.95 chars |
| Mean leaderboard literal length | 17.15 chars |

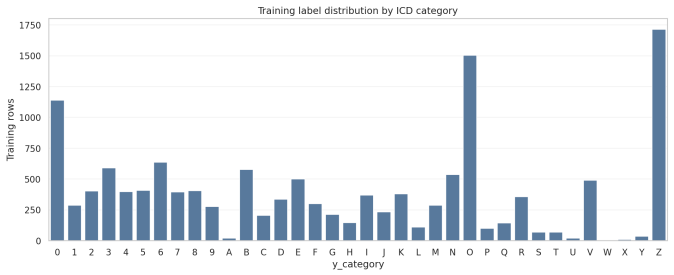


**Figure 1:** Training category distribution. The most frequent class is `Z`, followed by `0` and `O`.

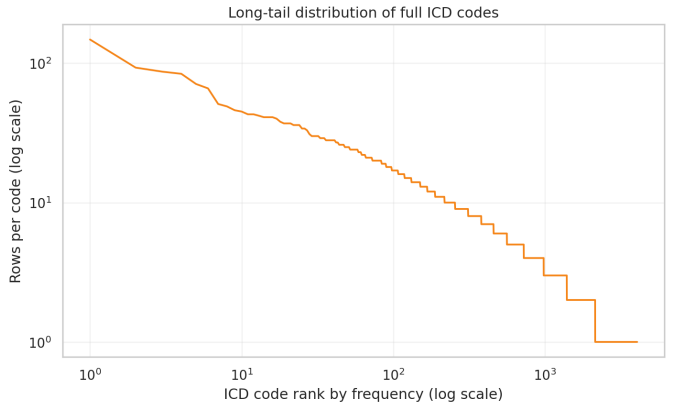


**Figure 2:** Long-tail distribution. Rare categories motivated macro-F1 reporting and imbalance-aware experiments.

## 4. Understanding the Main Challenges

The second phase was *Understanding the main challenges of the task*. After the first EDA pass, the data suggested five central difficulties.

**Class imbalance.** The majority baseline already reaches 0.125 accuracy by always predicting the most frequent category. Accuracy is therefore necessary but not sufficient; macro-F1 is important to see rare-class behavior.

**Clinical ambiguity.** A short literal may refer to different clinical contexts. Unlike long EMR-based ICD coding, this task rarely gives surrounding evidence.

**Abbreviations and symbols.** Literals include uppercase shorthand, slashes, hyphens, parentheses, digits, measurements, and accents. Removing them too aggressively could destroy signal.

**Broad target.** Predicting only the first ICD character is easier than full ICD coding, but it can hide clinically different codes inside the same broad prefix.

**Train–test shift.** We can compare literal lengths and surface patterns between train and leaderboard, but we do not observe leaderboard labels. Figure 3 shows that the token/length scale is similar, which reduces but does not remove distribution-shift concerns.

Figure 4 shows that the train and leaderboard sets share similar surface patterns. In training, 27.98% of literals contain accents, 11.83% are all-uppercase, 9.77% contain punctuation, and 7.98% contain digits. The leaderboard values are close: 29.59% accents, 11.31% all-uppercase, 8.95% punctuation, and 7.59% digits. This figure was useful because it converted a preprocessing intuition into evidence. If accents, punctuation and uppercase fragments appear consistently in both splits, then deleting them would not simply "clean" the text; it could remove information that the final model is expected to see at test time.

Together, Figures 3 and 4 also help interpret the later leaderboard results. They suggest that the public test literals are not obviously longer or structurally different from the training literals, so poor performance cannot be explained only by length shift. The remaining difficulty is more semantic: short clinical expressions may be compatible with several broad ICD prefixes.

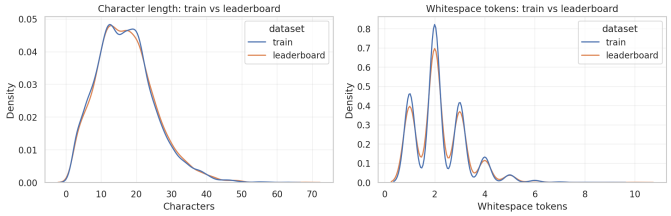


**Figure 3:** Train versus leaderboard literal lengths. Both splits contain short literals, but labels are unavailable for leaderboard rows.

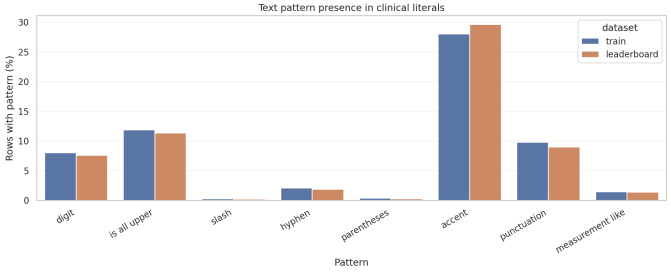


**Figure 4:** Presence of digits, uppercase literals, accents, punctuation, and other clinical-text patterns in train and leaderboard data.

## 5. Course and Literature Grounding

Yan et al. [2] describe the evolution of automated ICD coding from rule-based systems to traditional machine learning, neural models, knowledge-based methods, and pretrained language models. They also emphasize challenges that appeared in our own project: large label spaces, unbal-

anced labels, long clinical documents, and interpretability. Our assignment differs because the target is a single first-character category and the input is a literal rather than a full discharge summary, but the same core tensions remain: medical terminology, imbalance, ambiguity, and trust.

We also used Spanish clinical-coding context from CodiEsp [3]. This helped us place the assignment in a broader line of non-English clinical NLP work: even though our Kaggle task is smaller than full ICD coding, it still sits inside the same practical problem of turning clinical language into standardized codes.

The project connected directly to several course topics. The EDA phase relates to corpora and linguistic datasets. Preprocessing used Basic Text Processing ideas: regular expressions, whitespace normalization, tokenization, and the danger of losing linguistic information. TF–IDF baselines connect to vector-space models and feature engineering from Fundamentals of Machine Learning. RoBERTa connects to Transformers, self-attention, pretrained language models, and fine-tuning from NLP and Neural Networks and Deep Learning [4, 5, 6]. The final ensemble connects to the Machine Learning idea that different models may make partially uncorrelated errors.

Figure 5 summarizes how the survey's method evolution influenced our project decisions. We used it as a conceptual bridge between the course and the assignment: first we followed the classical NLP path with lexical and retrieval baselines, then we moved to pretrained biomedical Transformers, and finally we used an ensemble because in Machine Learning we had learned that combining diverse models can reduce model-specific errors when their mistakes are not perfectly correlated.

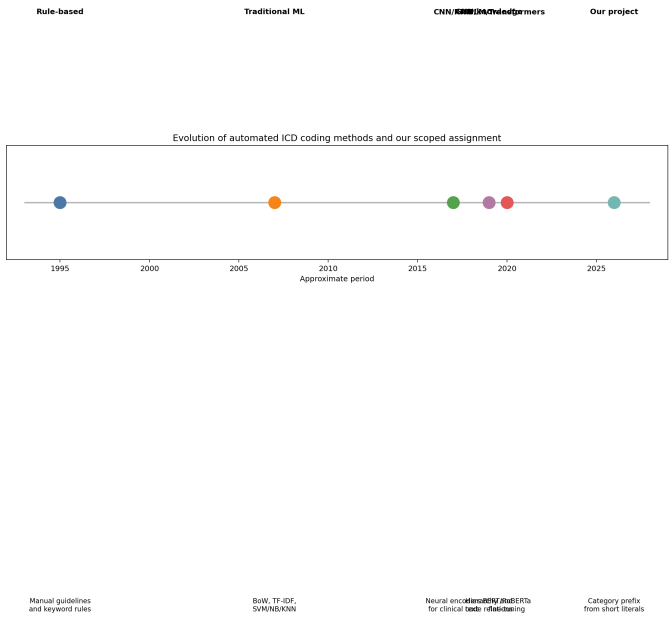


**Figure 5:** Method evolution from the ICD coding survey and how it guided our feasible project choices.

## 6. Preprocessing and Annotation Design

The final preprocessing pipeline is intentionally light:

```
text = str(text)
text = text.strip()
text = re.sub(r"\s+", "␣", text)
```

We did not lowercase, remove accents, or remove punctuation for the final RoBERTa pipeline. The reason is linguistic and practical: the backbone tokenizer was pretrained on Spanish biomedical and clinical text, and the EDA showed that case, accents, punctuation, digits, and abbreviations may encode medical meaning. This is why we moved from EDA to preprocessing cautiously. More aggressive normalization was treated as an ablation for classical models, not as the default.

Subword tokenization also influenced design. With `PlanTL-GOB-ES/roberta-base-biomedical-clinical-es`, the median token length was 5 tokens in both train and leaderboard, the 99th percentile was 12, and the maximum was 24 for training. We therefore selected a default `max_length` of 32. This is a small but important example of not treating Transformer input length as arbitrary: the value was chosen because the literals are genuinely short, not because 32 is a default hyperparameter. The diagnostic tokenization plots are kept in the appendix so that the main report remains readable.

## 7. Methodology

We implemented each model as a runnable versioned script that loads data, creates an 80/20 stratified split, evaluates validation predictions, writes metrics, writes detailed predictions, and generates a Kaggle submission.

**Majority baseline.** `v00` predicts the most frequent training-split class for every example. It is intentionally weak but essential: it tells us how much imbalance alone explains.

**Character TF–IDF.** `v01` uses `TfidfVectorizer(analyzer="char_wb")` with logistic regression. The EDA suggested this baseline because many useful clues live inside short words, abbreviations, fragments, and punctuation-heavy clinical literals.

**Word TF–IDF.** `v02` uses word n-grams with linear SVM/logistic alternatives. This tests whether whole-word lexical signal is enough.

**Similarity retrieval.** `v03` treats the task as nearest-neighbor retrieval over training literals using TF–IDF cosine similarity. It is intuitive and related to information-retrieval views of coding, but limited when similar strings have different labels.

**RoBERTa CLS.** `v04` fine-tunes a Spanish biomedical-clinical RoBERTa backbone. We moved to RoBERTa after the classical baselines showed that text signal existed, but sparse surface features could not fully resolve ambiguity. For CLS pooling:

$$\mathbf{h} = \mathbf{H}_{[:,0,:]}, \tag{2}$$

where the first token representation is passed through dropout and a linear classifier.

**RoBERTa mean pooling.** `v05` averages non-padding hidden states:

$$\mathbf{h} = \frac{\sum_{t=1}^{T} m_t \mathbf{H}_t}{\max(1, \sum_{t=1}^{T} m_t)}. \tag{3}$$

Mean pooling may help short literals because every token can contribute directly to the representation.

**Improvements.** The RoBERTa baselines improved accuracy, but rare classes and ambiguous literals remained fragile. This is why `v06` tested class-weighted cross entropy and focal loss, `v07` tuned learning rate, warmup, dropout, weight decay, batch size, gradient clipping, and AMP, and `v08` tested safe data strategies, especially duplicate handling and sampling. We avoided unsafe augmentation such

as random medical-word deletion or unverified synonym replacement.

**Ensemble.** `v09` combined complementary saved models. `v10` broadened the search toward diverse Machine Learning combinations. We chose the final ensemble following a principle studied in Machine Learning: an ensemble is most useful when its members are individually reasonable but make different kinds of errors. For that reason, the final public candidate, `v10_vote_diverse_no_retrieval`, combines RoBERTa safe-dedupe mean, RoBERTa CLS, character TF–IDF logistic regression, and word TF–IDF SVM. The two RoBERTa models provide biomedical contextual representations, while the TF–IDF models preserve strong surface-form evidence from abbreviations, fragments and short literals. Retrieval was useful for analysis, but it was not included in the final vote because ambiguous duplicate literals made nearest-neighbour evidence less reliable.

## 8. Training and Experimental Setup

All experiments used a stratified 80/20 train/validation split by `label_id`. Seeds were configurable and fixed in Python, NumPy, and PyTorch where applicable. Classical models were trained with scikit-learn [7]. RoBERTa models used the Hugging Face Transformers ecosystem [8] and the Spanish biomedical-clinical checkpoint from PlanTL-GOB-ES [9, 10].

Before filling the final report, we inspected `outputs/metrics`, `outputs/logs`, `EXPERIMENT_LOG.md`, `REPORT_NOTES.md`, `DECISIONS.md`, and `submissions`. Files containing `debug`, `dry_run`, or `smoke` were treated as infrastructure checks only and are not reported as final metrics. The results in Table 3 come from full runs or from the consolidated final evaluation tables.

For neural models, the main setup was cross entropy loss:

$$\mathcal{L} = -\frac{1}{N}\sum_{i=1}^{N} \log \frac{\exp z_{i,y_i}}{\sum_{c=1}^{C} \exp z_{i,c}}, \tag{4}$$

with AdamW, learning rate $2 \cdot 10^{-5}$, weight decay 0.01, dropout 0.1, early stopping by validation accuracy, and GPU acceleration when available in the course VM environment. No credentials are stored in the repository.

We report accuracy and F1. Accuracy is:

$$\mathrm{Acc} = \frac{1}{N}\sum_{i=1}^{N} \mathbb{1}[\hat{y}_i = y_i]. \tag{5}$$

For each class $c$:

$$F1_c = \frac{2P_cR_c}{P_c + R_c}. \tag{6}$$

Macro-F1 averages classes equally; weighted-F1 weights by class frequency. This distinction matters because the label distribution is imbalanced.

## 9. Results

Table 3 contains the main validation and public leaderboard results. The best public score verified in repository files is 0.587 for `v10_vote_diverse_no_retrieval`. The private/final ranking is not claimed.

Figure 6 shows why we did not skip the classical stage. The majority baseline is weak, but the character and word TF–IDF models immediately reach a much stronger validation region. This means that the literals contain strong lexical clues, and it also explains why the final ensemble keeps classical models instead of replacing them completely.

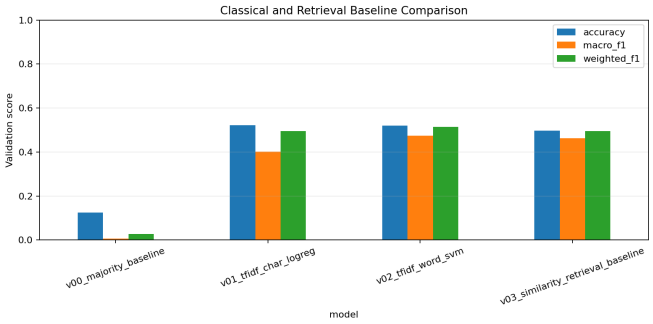


**Figure 6:** Classical and retrieval baseline comparison. The large jump from the majority baseline confirms that lexical evidence is informative.

Figure 7 visualizes the complete trend. RoBERTa improved further by adding contextual biomedical representations, and ensembles gave the best final candidates by combining both kinds of evidence. The result was not simply "deep learning beats everything". Character and word TF–IDF remained competitive and became valuable ensemble members. This matters for interpretation: in very short clinical literals, surface form still carries information that a Transformer may not always use in the same way.

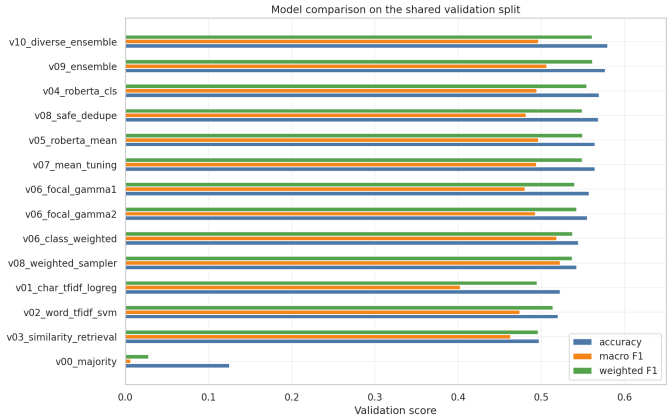


**Figure 7:** Validation comparison across model families. The ensemble improves because it combines contextual and lexical models.

The v10 search produced two important variants. The strongest validation-accuracy recipe was `vote_diverse_no_retrieval_val_weighted`, with accuracy 0.583577, macro-F1 0.501074, and weighted-F1 0.564988. The final public candidate was `vote_diverse_no_retrieval`, with validation accuracy 0.579562 and the best verified public score, 0.587. We report this distinction explicitly because internal validation and public leaderboard evidence are related but not identical.

The score should be interpreted carefully. A public score around 0.587 means that the system has learned useful structure from short literals, but it also means that many cases remain uncertain. This is consistent with the EDA: the inputs are very short, categories are imbalanced, and some repeated literals have conflicting labels. Therefore, the main achievement is not that the model is clinically deployable, but that the repository shows a coherent progression from data understanding to a justified final ensemble.

**Table 3:** Main model comparison. Public scores are reported only when a verified Kaggle submission record exists in the repository.

| Version | Model | Accuracy | Macro-F1 | Weighted-F1 | Public |
|---|---|---|---|---|---|
| v00 | Majority baseline | 0.125182 | 0.006181 | 0.027854 | – |
| v01 | Char TF–IDF + logreg | 0.522628 | 0.402554 | 0.494943 | 0.550 |
| v02 | Word TF–IDF + SVM | 0.520073 | 0.474196 | 0.514018 | 0.536 |
| v03 | Similarity retrieval | 0.497445 | 0.462789 | 0.496120 | – |
| v04 | RoBERTa CLS | 0.569343 | 0.494329 | 0.554347 | 0.573 |
| v05 | RoBERTa mean | 0.564599 | 0.496567 | 0.549541 | 0.556 |
| v06 | Imbalance-aware RoBERTa | 0.557299 | 0.480394 | 0.539776 | 0.555 |
| v07 | Tuned RoBERTa mean | 0.564599 | 0.494151 | 0.549054 | 0.572 |
| v08 | Safe data strategy | 0.568613 | 0.481648 | 0.549150 | 0.583 |
| v09 | Validation ensemble | 0.576642 | **0.506277** | **0.561544** | 0.573 |
| v10 | Diverse ensemble | **0.579562** | 0.496677 | 0.561294 | **0.587** |

## 10. Error Analysis and Interpretability

The evaluation notebook asks: *Which model should we trust as our final submission?* To answer it, we inspected confusion matrices, per-class recall, confidence distributions, and real examples. Figure 8 shows the final confusion matrix. Figure 9 shows that performance is uneven across categories, which is expected from the long tail.

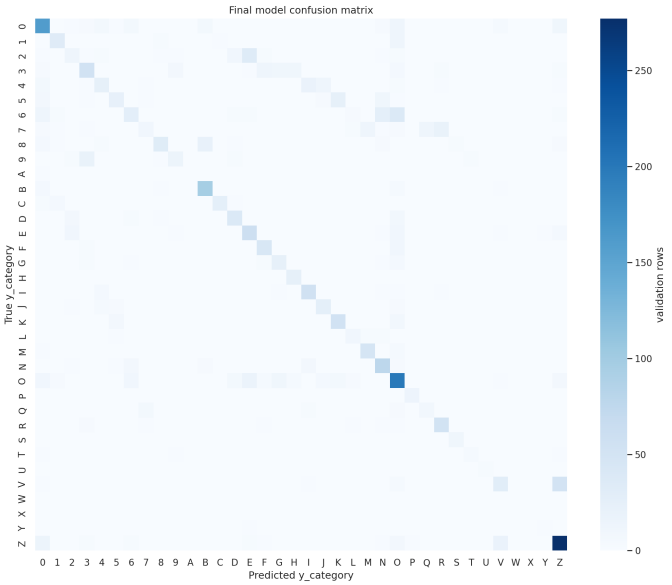


**Figure 8:** Confusion matrix for the final candidate. Confusions reflect broad ICD-prefix overlap and rare labels.

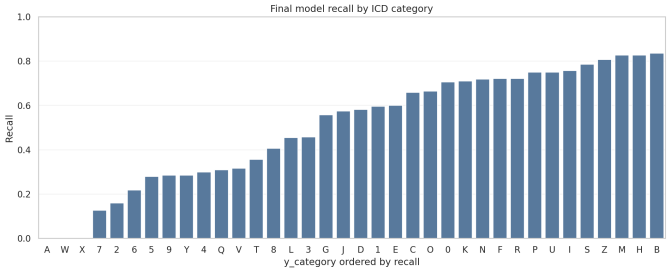


**Figure 9:** Per-class recall for the final candidate. Macro-F1 is important because frequent classes can hide weak rare-class behavior.

Confidence analysis in Figure 10 showed that high confidence does not guarantee correctness. The left plot separates correct and wrong top-1 confidence, while the right plot studies the margin between the first and second prediction. Correct predictions tend to have higher confidence and larger margins, but the overlap is still substantial. This is one of the most important practical findings of the project: confidence can be useful for triage, but it cannot be treated as a clinical explanation. Some wrong high-confidence examples are especially important in a hospital setting because they may look reliable to users. The typical causes of error were ambiguous literals, insufficient context, abbreviations, class imbalance, similar categories, and broad ICD-prefix granularity.

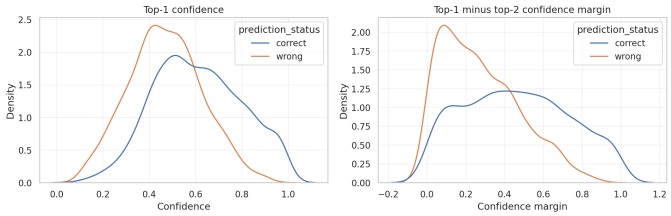


**Figure 10:** Confidence distributions for correct and incorrect predictions. Confidence is useful diagnostically, but not a clinical explanation.

We treat interpretability carefully. Confusion matrices, margins, nearest examples, and feature weights are useful diagnostics, but they are not proof of medical reasoning. If attention or token attribution is added later, it should be presented with limitations rather than as a faithful explanation of clinical causality.

The most frequent confusion pairs also made clinical sense at the surface level. Table 4 shows examples: `V` was often confused with `Z` for literals such as *prótesis mamarias* and *alergia Àcars*; `6` was confused with `O` for pregnancy-related literals such as *patología fetal parto*; and `2` was confused with `E` for endocrine/metabolic-looking literals such as *DIABETES*. We kept this table in the main report because it translates aggregate metrics into concrete error types. Figure 11 then visualizes the same idea at scale: the model does not fail randomly, but tends to confuse categories that share surface or clinical context. This helps explain why the ensemble improves the global score but cannot fully remove ambiguity.

## 11. GitHub Repository

The final repository is organised as a chain of notebooks rather than as isolated experiments. We kept this section compact because the report should explain the reasoning and results, while the repository should provide the executable detail. The notebooks are therefore not described one by one here; instead, Appendix A lists the full workflow in a format that does not overflow the two-column layout.

The repository is available at https://github.com/david06redrejo-bot/clinical-literal-icd-coding.

**Table 4:** Top observed confusion pairs for the final candidate.

| True | Pred. | Count | Example literals |
|---|---|---|---|
| V | Z | 51 | prótesis mamarias; alergia Àcars |
| 6 | O | 40 | patología fetal parto; preclampsia |
| 2 | E | 35 | DIABETES; déficit de ácido fólico |
| 6 | N | 26 | endometriosis; prolapso uteri |
| 5 | K | 24 | esteatosis hepática; hernia inguinal |

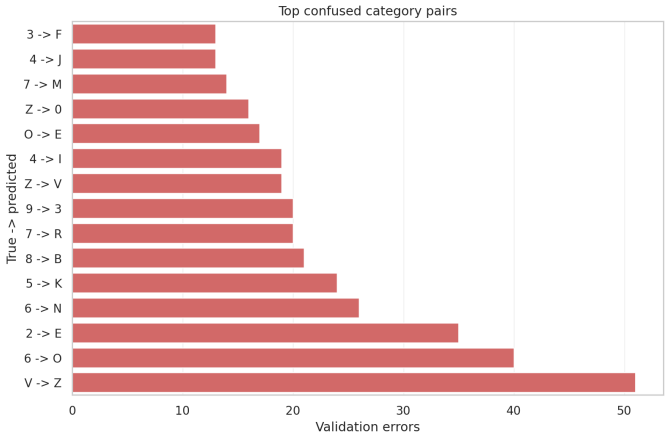


**Figure 11:** Top confused category pairs. The plot was generated from validation predictions, not from leaderboard labels.

It contains nine main notebooks covering task formulation and EDA, preprocessing and annotation design, classical baselines, retrieval and ICD catalogue analysis, biomedical RoBERTa, advanced experiments, ablations, error analysis, and final submission. This structure reflects how our group worked: first we understood the data, then we tested simple models, then we moved to pretrained language models, and finally we selected an ensemble only after seeing that different model families contributed complementary evidence.

This also makes the report traceable. The tables and figures are not independent decoration: they are the same evidence that the notebooks generate and inspect. For example, `reports/tables/final_experiment_comparison.csv` supports Table 3, `reports/tables/kaggle_submission_scores.csv` supports the public-score claim, and `submissions/final_submission.csv` is the final file submitted to Kaggle. In other words, the written report, the executed notebooks, and the GitHub repository describe the same workflow.

A practical limitation remains: the repository contains verified public Kaggle submissions, but not a verified private/final leaderboard screenshot or official ranking table. Therefore the report deliberately states the best verified public score and does not invent a private ranking claim. If final private leaderboard evidence becomes available, it should be added as a figure or table and referenced explicitly.

## 12. Final System and Kaggle Submission

The final submission file is:

```
submissions/final_submission.csv
```

The detailed leaderboard predictions are:

```
outputs/predictions/final_leaderboard_detailed.csv
```

The final Kaggle submission file contains exactly:

```
id, y_category
```

The best verified public score is 0.587 for `v10_vote_diverse_no_retrieval`. We do not state a final private ranking because no verified private leaderboard evidence is stored.

## 13. What Would Happen in a Hospital?

In a hospital, a system like this could help professional coders by suggesting candidate categories, highlighting uncertain cases, and making routine coding faster. The results show that this support would be useful but limited. A public score of 0.587 indicates that the model has learned meaningful patterns from short clinical literals, especially when lexical evidence and biomedical language representations agree. However, it also means that a substantial number of predictions are still wrong. In practice, this kind of system should therefore be used as a decision-support tool, not as an automatic coder.

The figures explain why. The class-distribution plots show that frequent categories dominate the training data, so the model is naturally more reliable for well-represented labels. The confidence plots show that wrong predictions can still appear confident, which means that confidence should be used to prioritize review rather than to bypass human validation. The confusion matrix and Table 4 show that many errors are not random: they happen between categories that can look similar from a short literal. This is exactly the kind of situation where a professional coder needs more context than the model receives.

The positive scenario is a human-in-the-loop workflow. The model could propose a first category, show the confidence margin, retrieve similar historical literals, and flag cases where the top alternatives are close. This could improve consistency in records, accelerate statistics, and support DRG-related administrative workflows. It could also help identify examples that require extra attention, especially when the model is uncertain or when the predicted category belongs to a rare class.

The risks are equally real. A wrong code may affect reimbursement, hospital management indicators, quality statistics, or downstream clinical data reuse. If users trust the model too much, high-confidence errors could become harmful. Privacy is also central: clinical text must be handled under strict governance, with access control and anonymization where appropriate. Bias is another risk: if some categories, services, or language styles are underrepresented, the model may perform worse for them. For these reasons, the system should support professionals, not replace them. Interpretability tools should help review decisions, but human accountability remains necessary.

## 14. Limitations

First, the target is only the broad ICD prefix, not the full ICD code. This makes the competition tractable but less clinically specific than real coding. Second, the inputs are short literals, so many examples lack the context needed for confident coding. Third, we did not implement a full ICD hierarchy model, GNN, knowledge graph, or label-description matching system, although the ICD catalogue file makes some future work possible. Fourth, we did not

compare with podium teams' models or ensemble with other teams' predictions because those models were not available in the repository. Fifth, public leaderboard feedback is not a substitute for a final private leaderboard result.

## 15. Future Work

Future work follows directly from these limitations and from the figures discussed in the report. Since the long-tail plots show that many codes are rare, future models should use ICD descriptions and hierarchy more explicitly instead of relying only on observed examples. Since the confidence plots show overlap between correct and wrong predictions, calibration and abstention thresholds should be studied before any human-facing deployment. Since the confusion analysis shows structured errors, a retrieval-augmented interface could present similar historical literals and ICD descriptions together with the predicted category. We could also predict full ICD codes rather than only prefixes, model hierarchy through parent–child relations or graph neural networks, and compare with other teams' methods if allowed. Any augmentation should remain clinically safe: we would avoid random deletion, unverified synonym replacement, or changes that affect negation.

## 16. Conclusion and Reflection

This project became more than a leaderboard exercise. We started with a basic baseline and a short presentation, received feedback that our direction was reasonable, and then built a complete NLP workflow around the data. The final result was a diverse ensemble with the best verified public score in our repository, but the deeper value was understanding why each step mattered. The figures were essential in that process: they showed imbalance, train–test similarity, preprocessing risks, baseline strength, Transformer behaviour, confidence limitations and structured errors.

We are grateful for the practical activity because it grounded course concepts in a real application. Text processing was not just regex; it was the decision not to destroy accents, punctuation, and abbreviations. Transformers were not just architecture diagrams; they were a way of reusing biomedical Spanish language knowledge. Evaluation was not just a number; it was a way of seeing which clinical categories and examples remained fragile.

Language is part of human life, and in healthcare it becomes part of memory, organization, payment, and care. That makes NLP powerful, but also responsible. Our final system is not ready for clinical deployment, but it is a traceable academic prototype showing how classical NLP and pretrained biomedical language models can work together on ICD category codification.

## A. Notebook Workflow

The repository notebooks are intended to be read as a sequence. They are listed here in a compact code block so that long filenames do not overflow the two-column layout.

```
00_task_formulation_and_eda.ipynb
01_data_preprocessing_and_annotation_design.ipynb
02_classical_baselines_and_vector_space_models.ipynb
03_similarity_retrieval_and_icd_catalogue_analysis.ipynb
04_biomedical_roberta_baselines.ipynb
05_advanced_model_experiments.ipynb
06_hyperparameter_and_ablation_studies.ipynb
07_evaluation_error_analysis_and_interpretability.ipynb
08_submission_and_final_story.ipynb
```

The main report keeps the narrative focused on decisions and results. The notebooks provide the implementation details: paths, data loading, label construction, preprocessing functions, tokenization checks, training scripts, metrics, figures, predictions and submission formatting.

## B. Additional Diagnostic Figures

This appendix collects supporting plots that are useful for reproducibility and diagnosis, but would interrupt the main argument if placed in the central narrative.

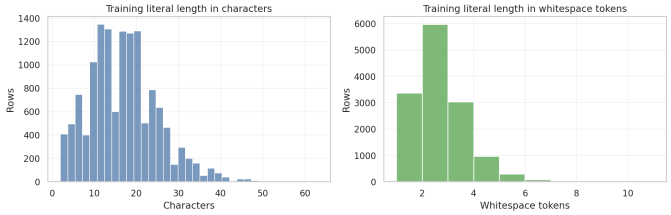


**Figure 12:** Training literal length distributions. The short length of most literals explains why surface features and subword tokenization are both important.

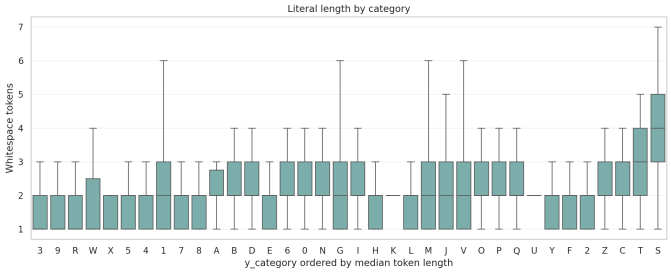


**Figure 13:** Literal length by category. Some categories tend to have slightly longer literals, but length alone is not enough to solve the task.

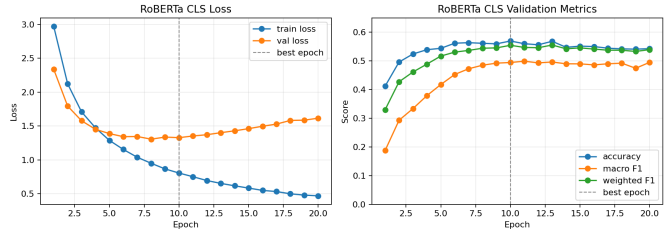


**Figure 14:** RoBERTa CLS training curves. Validation metrics improve quickly and then plateau, while validation loss later increases, supporting early stopping.

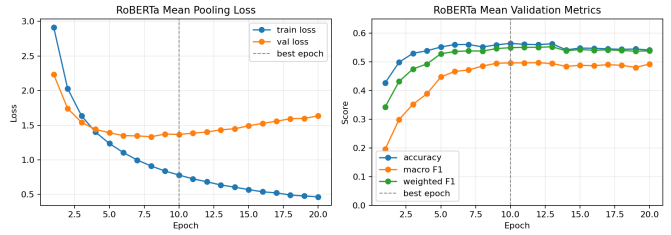


**Figure 15:** RoBERTa mean-pooling training curves. The behaviour is similar to CLS pooling, which explains why both representations were useful ensemble members.

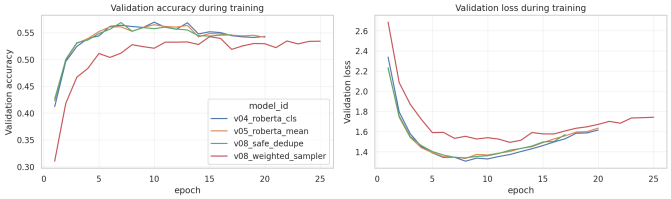


**Figure 16:** Training curves for selected RoBERTa variants. The curves show that several variants converge to a similar validation region, so the final decision required validation metrics, public score evidence and error analysis.

## Appendix: Reproducibility Commands

```
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt

python scripts/analyze_data_annotations.py
python scripts/run_preprocessing.py
python scripts/analyze_tokenization.py

python models/v00_majority_baseline.py
python models/v01_tfidf_char_logreg.py
python models/v02_tfidf_word_svm.py
python models/v03_similarity_retrieval_baseline.py

python models/v04_roberta_cls.py
python models/v05_roberta_mean.py
python models/v06_roberta_mean_imbalance_aware.py
python models/v07_roberta_mean_tuning.py
python models/v08_roberta_mean_augmented.py
python models/v09_ensemble.py
python models/v10_diverse_ensemble_search.py

python -m src.evaluation

kaggle competitions submit \
  -c uab-asho-ai-codification \
  -f submissions/final_submission.csv \
  -m "Team␣10␣final␣public-best␣v10␣diverse␣ensemble"
```

## References


[1] Kaggle. *UAB-ASHO AI Codification Competition*. https://www.kaggle.com/competitions/uab-asho-ai-codification.

[2] Chenwei Yan, Xiangling Fu, Xien Liu, Yuanqiao Zhang, Yue Gao, Ji Wu, and Qiang Li. A Survey of Automated International Classification of Diseases Coding: Development, Challenges, and Applications. *Intelligent Medicine*, 2022.

[3] Antonio Miranda-Escalada, Aitor Gonzalez-Agirre, Jordi Armengol-Estapé, and Martin Krallinger. Overview of automatic clinical coding: annotations, guidelines, and solutions for non-English clinical cases at CodiEsp track of eHealth CLEF 2020. *CLEF 2020 Working Notes*, 2020.

[4] Ashish Vaswani et al. Attention Is All You Need. *Advances in Neural Information Processing Systems*, 2017.

[5] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *NAACL-HLT*, 2019.

[6] Yinhan Liu et al. RoBERTa: A Robustly Optimized BERT Pretraining Approach. *arXiv preprint arXiv:1907.11692*, 2019.

[7] Fabian Pedregosa et al. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 2011.

[8] Thomas Wolf et al. Transformers: State-of-the-Art Natural Language Processing. *EMNLP: System Demonstrations*, 2020.

[9] Casimiro Pio Carrino, Jordi Armengol-Estapé, Asier Gutiérrez-Fandiño, Joan Llop-Palao, Marc Pàmies, Aitor Gonzalez-Agirre, and Marta Villegas. Pretrained Biomedical Language Models for Clinical NLP in Spanish. *Proceedings of the 21st Workshop on Biomedical Language Processing*, 2022.

[10] PlanTL-GOB-ES. *roberta-base-biomedical-clinical-es*. Hugging Face model checkpoint.